

PROGRAM 1

FLOWER IMAGE CLASSIFICATION USING CUSTOM CNN

OBJECTIVE

To develop a Convolutional Neural Network (CNN) model that classifies flower images into different categories.

In this program students will learn:

- ✓ Reading dataset from folders
- ✓ Image preprocessing
- ✓ CNN model creation
- ✓ Model training
- ✓ Model evaluation
- ✓ Accuracy and Loss curves
- ✓ Image prediction

This program helps in solving:

Q1, Q3, Q5, Q7, Q8, Q9, Q10, Q11, Q12 and Q15

DATASET FOLDER STRUCTURE

Flowers/

| — Daisy/

| | — image1.jpg

| | — image2.jpg

```
| └─ ...  
  
|  
  
└─ Rose/  
  | └─ image1.jpg  
  | └─ image2.jpg  
  | └─ ...  
  |  
└─ Sunflower/
```

```
└─ image1.jpg  
└─ image2.jpg  
└─ ...
```

Each folder name automatically becomes a class label.

STEP 1 IMPORT LIBRARIES

Purpose:

Import all required packages.

Code:

```
import tensorflow as tf  
import numpy as np  
import matplotlib.pyplot as plt  
  
from tensorflow.keras.preprocessing.image import ImageDataGenerator  
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Conv2D  
from tensorflow.keras.layers import MaxPooling2D  
from tensorflow.keras.layers import Flatten  
from tensorflow.keras.layers import Dense
```

```
from tensorflow.keras.layers import Dropout

print("Libraries Imported Successfully")
```

Explanation:

TensorFlow → Deep Learning Library

NumPy → Numerical Operations

Matplotlib → Plotting Graphs

ImageDataGenerator → Reading Images from Folder

STEP 2 LOAD DATASET

Purpose:

Read images directly from folders.

Code:

```
datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)

train_data = datagen.flow_from_directory(
    "Flowers",
    target_size=(128,128),
    batch_size=32,
    class_mode='categorical',
    subset='training'
)

val_data = datagen.flow_from_directory(
    "Flowers",
    target_size=(128,128),
    batch_size=32,
    class_mode='categorical',
    subset='validation'
)
```

```
print("Dataset Loaded Successfully")
```

Explanation:

rescale=1./255

Normalizes image values from:

0-255

to

0-1

validation_split=0.2

20% data used for validation

target_size=(128,128)

Resizes every image to 128×128

STEP 3 DISPLAY SAMPLE IMAGES

Purpose:

Verify dataset is loaded correctly.

Code:

```
images, labels = next(train_data)

plt.figure(figsize=(10,5))

for i in range(6):
    plt.subplot(2,3,i+1)
    plt.imshow(images[i])
    plt.axis("off")

plt.show()
```

Explanation:

Displays six random images from dataset.

STEP 4 CHECK CLASS LABELS

Purpose:

Identify class names.

Code:

```
print(train_data.class_indices)
```

Expected Output:

```
{  
  'Daisy':0,  
  'Rose':1,  
  'Sunflower':2  
}
```

STEP 5 BUILD CNN MODEL

Purpose:

Create Deep Learning Architecture.

Code:

```
model = Sequential([  
  
  Conv2D(  
    32,  
    (3,3),  
    activation='relu',  
    input_shape=(128,128,3)  
  ),
```

```
MaxPooling2D(),

Conv2D(
64,
(3,3),
activation='relu'
),

MaxPooling2D(),

Flatten(),

Dense(
128,
activation='relu'
),

Dropout(0.3),

Dense(
3,
activation='softmax'
)

])
```

Explanation:

Conv2D

Extracts image features.

Examples:

Edges

Shapes

Textures

MaxPooling

Reduces image dimensions.

Flatten

Converts matrix into vector.

Dense

Performs classification.

Softmax

Produces class probabilities.

STEP 6 VIEW MODEL ARCHITECTURE

Code:

```
model.summary()
```

Explanation:

Shows:

Number of Layers

Output Shapes

Trainable Parameters

STEP 7 COMPILE MODEL

Purpose:

Configure learning process.

Code:

```
model.compile(  
optimizer='adam',  
loss='categorical_crossentropy',
```

```
metrics=['accuracy']  
)
```

Explanation:

Adam

Most commonly used optimizer.

categorical_crossentropy

Used for multiclass classification.

STEP 8 TRAIN MODEL

Purpose:

Learn patterns from images.

Code:

```
history = model.fit(  
    train_data,  
    validation_data=val_data,  
    epochs=5  
)
```

Explanation:

Epoch = One complete pass through dataset.

STEP 9 EVALUATE MODEL

Purpose:

Check model performance.

Code:


```
loss, accuracy = model.evaluate(val_data)

print("Accuracy =", accuracy)
```

STEP 10 PLOT ACCURACY CURVE

Purpose:

Visualize learning.

Code:

```
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])

plt.title("Accuracy Curve")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")

plt.legend([
    'Training',
    'Validation'
])

plt.show()
```

STEP 11 PLOT LOSS CURVE

Code:

```
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])

plt.title("Loss Curve")
plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.legend([
```

```
'Training',  
'Validation'  
)  
  
plt.show()
```

STEP 12 PREDICT A NEW IMAGE

Purpose:

Classify unseen image.

Code:

```
from tensorflow.keras.preprocessing import image  
  
img = image.load_img(  
    "test.jpg",  
    target_size=(128,128)  
)  
  
img_array = image.img_to_array(img)  
  
img_array = img_array/255.0  
  
img_array = np.expand_dims(  
    img_array,  
    axis=0  
)  
  
prediction = model.predict(  
    img_array  
)  
  
print(prediction)
```

WHAT STUDENTS LEARN

✓ Folder Dataset Reading

✓ Image Preprocessing

✓ CNN Architecture

✓ Conv2D

✓ ReLU

✓ MaxPooling

✓ Flatten

✓ Dense Layer

✓ Dropout

✓ Adam Optimizer

✓ Model Training

✓ Model Evaluation

✓ Accuracy Curve

✓ Loss Curve

✓ Image Prediction

EXPECTED VIVA QUESTIONS

1. Why is normalization required?
2. Why is ReLU used?
3. What happens if pooling is removed?
4. Why is Softmax used?
5. What is the role of Dropout?
6. What happens if epochs increase?
7. What is overfitting?
8. What is underfitting?

9. Why is Adam optimizer used?

10. How can accuracy be improved?